

Assignment 2

DUE DATE: 11:59PM MELBOURNE TIME, MONDAY APRIL 27TH, 2026

1 Introduction

This handout is the Assignment 2 sheet. The assignment is worth 20% of your total mark. You will carry out the assignment in the same pairs as for Assignment 1, *unless you were allocated to a new pair in case your partner withdrew from the subject.*

In this assignment you will use Alloy to diagnose and investigate the root cause and fix for a March 2023 data leakage vulnerability in the ChatGPT web application.

This vulnerability resulted in some users' HTTP requests to the ChatGPT web application returning HTTP responses containing the data of other users, e.g., their chat histories, etc.

The vulnerability was real, but we will consider a simplified model of the ChatGPT web application, inspired by OpenAI's write-up of the incident¹. In this model, the ChatGPT web application comprises two components: the (Python) Web Application front-end and a Redis Cluster back-end. The intended interactions of these components is depicted in Fig. 1.

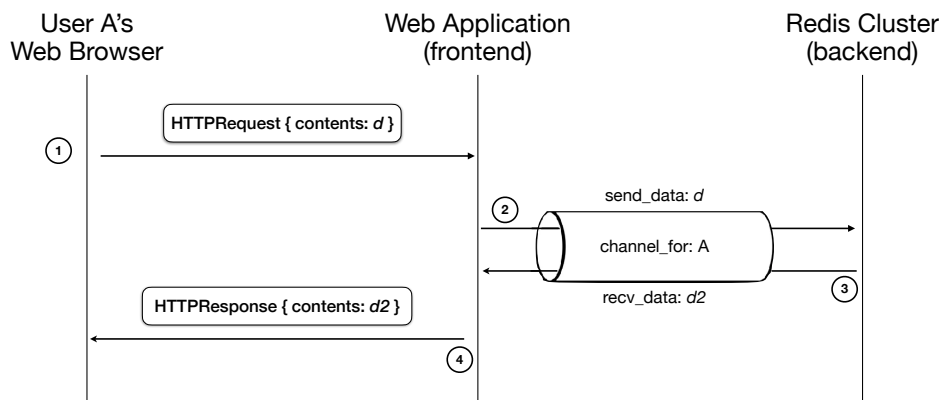


Figure 1: Example intended interactions in the protocol.

1.1 Communications Media

There are two kinds of communications media that are used in this system. Firstly, there is the *HTTP network* on which HTTP messages travel between users' web browsers and the the web frontend. Secondly, the communications between the web frontend and Redis backend occur via *Connections* allocated from a pool. Fig. 1 depicts one such Connection as a cylinder carrying data between the web front- and backends.

¹<https://openai.com/index/march-20-chatgpt-outage/>

1.2 No Adversary

For the purposes of this assignment, we assume that all communications media are free of malicious attackers. The vulnerability we are investigating does not require a malicious attacker to trigger it.

1.3 Connections and Cancellation

While there is no explicit adversary in the scenario that we investigate, there *are* exceptional conditions. Poor resource handling under exceptional conditions is a well-known source of errors in real-world systems [1, 2], and this case study is no different.

In particular, when receiving an HTTPRequest message, to process that message the web frontend needs to allocate a Connection to communicate with the Redis Cluster. HTTPRequests contain a “src” field to record which User sent the message. In this simplified scenario, the web frontend maintains a table called “connection_for” that maps from Users to Connections that records the connection currently allocated for each User.

Each connection we model as a pair of uni-directional channels: the “send_data” channel carries data from the web frontend to the backend, while the “recv_data” channel carries data in the reverse direction. We sometimes also refer to these channels as *buffers*, e.g., we might refer to a Connection’s “send_data” buffer, etc.

Finally, the web frontend can decide to cancel a previously-received user’s HTTP request. Doing so frees up the corresponding entry in the “connection_for” table.

1.4 End-to-End User Request Handling

With these ingredients, we can now explain the interaction sequence depicted in Fig. 1. First, in step ①, a User sends an HTTPRequest onto the network containing some data d . In step ②, the web frontend receives that request, allocates a Connection to communicate with the Redis Cluster backend and writes the data d to the Connection’s “send_data” buffer. In step ③, the Redis Cluster backend responds with data $d2$, writing it to the Connection’s “recv_data” buffer. In step ④ the web application frontend reads that data, releases the Connection that was previously allocated, and sends back an HTTPReply to the original User containing the data $d2$.

1.5 System State

The system state comprises:

http_network carries HTTP messages (either HTTPRequests or HTTPResponses). Only one message can be carried on the network at a time (to keep the model tractable).

connection_for stores for each Connection whether that connection is currently allocated to a User or not (and which User if so).

connection_send_data stores for each Connection up to a single item of user data, modelling the connection’s “send_data” buffer (for data sent from the web frontend to the backend).

connection_recv_data stores for each Connection up to a single item of user data, modelling the connection’s “recv_data” buffer (for data sent from the web backend to the frontend).

last_action is for debugging and identifies which was the last action to occur (see below).

User Data Each User has a set of data items that belong to them. No two users share any data items, i.e., if data d belongs to user u , then d does not belong to any other user $u2 \neq u$. The web frontend also uses a special sentinel data value, **DataRequestCancelled**, to inform a user that their request was cancelled. This sentinel value does not belong to any user.

1.6 Actions

The following actions can occur in the system:

User Sends an HTTPRequest , by placing an HTTPRequest on to the http_network. The message’s “src” field identifies the User who sent it.

User Receives an HTTPResponse , removing it from the http_network.

HTTPRequest is received and Connection acquired , occurs when the web frontend receives an HTTPRequest from the http_network, removes it from the network, acquires an unallocated Connection for the “src” User of the HTTPRequest, recording that the connection was allocated in the “connection_for” table, and writing the HTTPRequest’s “contents” data to the acquired Channel’s “send_data” buffer.

Redis Cluster processes a Connection , which occurs when there is data in a Connection’s “send_data” buffer, in which case the data is removed from that buffer and some other data is written to the same channel’s “recv_data” buffer. The data written to the “recv_data” buffer belongs to the same user to which the data in the channel’s “send_data” buffer belonged. Put another way, the Redis Cluster responds to data owned by some user with data also belonging to that same user.

Connection is released and HTTPResponse sent occurs when there is data in a Connection’s “recv_data” buffer. The web frontend reads that data, creates an HTTPResponse with the data as its contents, destined for the User to whom the Connection is currently allocated, places the HTTPResponse on the http_network (which must be empty), deallocates the Connection (removing the corresponding entry from the “connection_for” table), and clears the Connection’s “recv_data” buffer.

User request is Cancelled can occur when a Connection is currently allocated to some User. The web frontend deallocates the Connection (removing the corresponding entry from the “connection_for” table) and sends an HTTPResponse to that User containing the **DataRequestCancelled** sentinel value. This action can only occur when the http_network is empty.

1.7 Security Goal

The security goal of this system is that the web frontend should never send an HTTP message onto the http_network that is destined for some user u but contains the user data of another

user $u2$ where $u2 \neq u$.

2 The Formal Model

You are provided with a partial Alloy model of this system in the file `redis.als`. Part of your tasks for this assignment is to complete this model using the description above and the following information.

2.1 Data and Users

We model data using an abstract type `Data`. The subtype `UserData` represents data that belongs to specific users, while `DataRequestCancelled` is a special sentinel value used to inform a user that their request was cancelled.

```
abstract sig Data {}
sig UserData extends Data {}
one sig DataRequestCancelled extends Data {}
```

Each `User` has a set of `UserData` items that belong to them, captured by the `my_data` field. You will need to write a fact to ensure that no two users share any data items.

```
sig User {
  my_data : set UserData
}
```

2.2 HTTP Messages

HTTP messages carry `Data` as their contents. An `HTTPRequest` records the user who sent it (`src`), while an `HTTPResponse` records the user to whom it is destined (`dest`).

```
abstract sig HTTPMessage {
  contents : Data
}

sig HTTPResponse extends HTTPMessage {
  dest : User
}

sig HTTPRequest extends HTTPMessage {
  src : User
}
```

2.3 Connections

Connections model the Redis connection pool. Each connection can be allocated to at most one user at a time and has separate send/receive data buffers for communicating with the Redis backend.

```
sig Connection {}
```

2.4 System State

The Alloy model defines a **State** type that captures the entire system state as a set of time-varying (**var**) fields.

```
one sig State {  
  var http_network : lone HTTPMessage,  
  var connection_send_data : Connection -> lone UserData,  
  var connection_recv_data : Connection -> lone UserData,  
  var connection_for : Connection -> lone User,  
  var last_action : Action  
}
```

The state fields correspond to the system state described earlier in this document. The **last_action** field is provided for debugging: it records which action was last performed, to make it easier to interpret traces produced by Alloy. The correspondence between **Action** values and action predicates is as follows:

Action value	Predicate
DoNothing	do_nothing
UserSendReq	action_user_send_http_request
UserRecvResp	action_user_recv_http_response
RecvReqAcquireConn	action_recv_http_request_and_acquire_connection
RedisProcess	action_redis_process_connection
ReleaseConnSendResp	action_release_connection_and_send_http_response
RequestCancelled	action_user_request_cancelled

2.5 The BugFixed Toggle

The model includes a **BugFixed** sig that acts as a toggle:

```
lone sig BugFixed {}
```

When **BugFixed** is present (i.e. **some BugFixed**), the bug fix is enabled. When **BugFixed** is absent (i.e. **no BugFixed**), the system exhibits the vulnerable behaviour. You will use this mechanism in Task 4 to guard your bug fix, so that the same Alloy file can model both the vulnerable and the fixed system.

2.6 Given Predicates

The following predicates are provided to you in the template. You should study them to understand how the model works.

do_nothing This predicate models a stuttering step in which no action occurs. All state fields remain unchanged and **last_action** is set to **DoNothing**.

Transition and initialisation facts. The `Init_is_Initial` fact constrains the initial state. The `trans` fact constrains every step to be one of the defined action predicates (including `do_nothing`).

You should not modify any of the given predicates or facts.

2.7 Predicates You Will Complete

Each action predicate you write must update the `last_action` field to the corresponding `Action` value (see the table above). Each action predicate must also include *frame conditions*: explicit constraints on the state fields that the action does *not* change. If a field is not mentioned in a predicate's effect, Alloy is free to assign it any value in the next state, which will produce spurious behaviours.

Hint: the `action_user_send_http_request` predicate should constrain that the `HTTPRequest`'s contents are a member of the sending `User`'s `my_data` set. This is important for the model's security reasoning: it ensures that only a user's own data appears in `HTTP` requests that they send.

Hint: the `Redis Cluster` does not know which user data belongs to. However, it always responds with data that belongs to the same user as the data it received. We model this via a helper predicate `user_data_for_same_user[d, d2 : UserData]` that you will write. This predicate should hold whenever `d` and `d2` belong to the same user.

Hint: when a `Connection` is acquired in `action_recv_http_request_and_acquire_connection`, the only requirement is that the `Connection` is not currently allocated to any `User` (i.e. has no entry in `connection_for`). Be careful not to over-constrain this predicate with additional preconditions beyond what is stated here.

3 Your Tasks

*You should not add any additional **sig** declarations nor modify any of the existing **sig** declarations during this assignment. You should not add or remove parameters to the **pred** declarations during this assignment.*

3.1 Task 1: Complete the Model (9 marks)

Your first task is to complete the parts of the model that are missing. Doing so will require you to carefully understand the model you are given so you can work out how to fill in the missing parts.

- a) [1 mark] **User data disjointness.** Write a predicate `user_data_disjoint` that expresses the property that no two different users share any data items. Test your predicate by writing suitable **run** or **check** commands to convince yourself that it correctly captures the intended property. Then promote your predicate to a **fact**, so that your file contains something like:

```
fact user_data_disjoint {
```

```
// implementation goes here
}
```

- b) [0.5 marks] **Init.** Complete the `Init` predicate. In the initial state, all variable state fields should be empty and the `last_action` should be `DoNothing`.
- c) [1 mark] **action_user_send_http_request.** Complete this predicate. A User sends an `HTTPRequest` onto the `http_network`. The network must be empty beforehand. The request's contents must be some data from the sending User's `my_data` set.
- d) [0.5 marks] **action_user_rcv_http_response.** Complete this predicate. A User receives an `HTTPResponse` that is currently on the `http_network` and is destined for that User. The message is removed from the network.
- e) [1.5 marks] **action_rcv_http_request_and_acquire_connection.** Complete this predicate. The web frontend receives an `HTTPRequest` from the `http_network` and removes it from the network, acquires a `Connection` that is not currently allocated to any User, records in `connection_for` that the `Connection` is now allocated to the request's sender, and writes the request's contents to the `Connection`'s `connection_send_data` buffer.
- f) [1.5 marks] **user_data_for_same_user** and **action_redis_process_connection.** First, complete the helper predicate `user_data_for_same_user[d, d2 : UserData]`. This predicate should hold when `d` and `d2` belong to the same user.

Then complete `action_redis_process_connection`. This predicate models the Redis backend processing a `Connection` that has data in its `connection_send_data` buffer: it removes that data from the buffer and writes some (possibly different) data belonging to the same user to the `Connection`'s `connection_rcv_data` buffer (which must have been empty beforehand).

- g) [1.5 marks] **action_release_connection_and_send_http_response.** Complete this predicate. When a `Connection` has data in its `connection_rcv_data` buffer, the web frontend reads that data, creates an `HTTPResponse` with the data as its contents destined for the User to whom the `Connection` is currently allocated, places the `HTTPResponse` on the `http_network` (which must be empty beforehand), deallocates the `Connection` (removing the corresponding entry from `connection_for`), and clears the `Connection`'s `connection_rcv_data` buffer.
- h) [1.5 marks] **action_user_request_cancelled.** Complete this predicate. When a `Connection` is currently allocated to some User, the web frontend deallocates the `Connection` (removing the corresponding entry from `connection_for`) and sends an `HTTPResponse` to that User containing the `DataRequestCancelled` sentinel value. This action can only occur when the `http_network` is empty.

3.2 Task 2: Discover the Vulnerability (3 marks)

- a) [1 mark] **NoDataLeak assertion.** Write an assertion `NoDataLeak` that asserts that in all states the system might reach, the security goal of Section 1.7 holds. Write a **check** command with a suitable bound. You should find that the assertion is *violated*.

- b) [2 marks] **Vulnerability trace.** Write a `run` command called `vulnerability` that demonstrates the data leakage vulnerability. Your `run` command should encode a specific sequence of actions (using Alloy's sequential composition operator `;`) that leads to a state in which the `NoDataLeak` property is violated. Add comments above your `run` command explaining the sequence of events and *why* the vulnerability arises.

3.3 Task 3: Diagnose the Root Cause (3 marks)

- a) [2 marks] **Invariant.** Write a predicate `inv` that captures an invariant of the system that *should* always hold: specifically, a property relating unallocated Connections to the contents of their data buffers. Write a `check` command to verify whether your invariant holds in all states that can be reached in the vulnerable system (i.e. when `no BugFixed`). You should find that it is violated.

Hint: think about what property a correctly-functioning connection pool should maintain about connections that are not currently allocated to any user.

- b) [1 mark] **Invariant violation.** Write a comment explaining: (i) which action predicate causes the invariant to be violated and what that action predicate fails to do that it should, and (ii) how the resulting invariant violation enables the data leakage vulnerability you found in Task 2.

3.4 Task 4: Fix and Verify (5 marks)

- a) [2 marks] **Implement the fix.** Using your analysis from Task 3, modify the action predicate that is the root cause of the vulnerability to fix the problem. Use the `BugFixed` sig as a guard (see Section 2.5): when `no BugFixed`, the predicate should behave as before (vulnerable); when `some BugFixed`, the predicate should implement the fix.

Hint: use `no BugFixed implies (...)` and `some BugFixed implies (...)` to guard the two branches. Be careful with Alloy's operator precedence: `and` binds more tightly than `implies`, so you will need parentheses. For example:

```
(no BugFixed implies (  
  // ... vulnerable behaviour here ...  
)) and  
(some BugFixed implies (  
  // ... fixed behaviour here ...  
))
```

- b) [1 mark] **Verify the fix.** Write `check` commands to verify that, when `some BugFixed`:
- the `NoDataLeak` assertion holds, and
 - your `inv` invariant holds in all reachable states.
- c) [2 marks] **Discussion.** Add comments to the bottom of your Alloy file discussing the following two topics.
- (i) [1 mark] **Bounds.** Justify your choice of bounds for the verification checks in Task 4b. What classes of system behaviours are covered by those bounds? What

confidence does a successful check provide about the fixed system, and what are the inherent limitations of bounded verification?

- (ii) [1 mark] **Model limitations.** Identify at least one simplification or abstraction in the Alloy model that could mean a real-world vulnerability goes undetected, and explain *concretely* what kind of vulnerability or behaviour it could miss. A good answer will identify a specific scenario that the model cannot capture, not just observe that the model is an abstraction.

Use of Generative AI

You are permitted to use Generative AI tools to assist you with this assignment. However, you should be aware of the following:

- **The final exam will require you to read and write Alloy.** The purpose of this assignment is to build your understanding of formal modelling with Alloy, including temporal specifications, frame conditions, invariants, and bounded verification. If you rely on Generative AI to produce your Alloy code without understanding what it does and why, you will be at a significant disadvantage in the exam.
- **You remain responsible for the content of your submission.** Any errors that a Generative AI introduces are *your* errors. Alloy has subtle semantics (e.g. operator precedence, quantifier scoping, the distinction between `+` and `++`) that current LLMs frequently get wrong. You must understand and verify any code you submit.
- **Validate AI-generated code using Alloy.** If you use Generative AI to help produce Alloy code, do not simply accept its output. Use Alloy's own facilities — **run** and **check** commands — to validate that the generated code behaves as intended. For example, write **run** commands to confirm that predicates are satisfiable, or **check** commands to verify expected properties. This is good practice regardless of whether you use AI.
- **Document your use of Generative AI.** If you use Generative AI, add a brief comment to the bottom of your Alloy file describing how you used it, and critically, how you validated that the AI-generated code was correct (e.g. what **run**/**check** commands you used and what they told you).

Academic Misconduct

The University misconduct policy applies to this assignment. Students are encouraged to discuss the assignment topic, but all submitted work must represent the pair's understanding of the topic.

The subject staff take plagiarism very seriously. In the past, we have successfully prosecuted several students that have breached the university policy. Often this results in receiving 0 marks for the assessment, and in some cases, has resulted in failure of the subject.

Submission

Submit your Alloy file using the link on the subject LMS.

Only *one* student from the pair should submit the solutions, and each submission should clearly identify **both** authors.

Late submissions Late submissions will attract a penalty of 10% (2 marks) for every day that they are late. If you have a reason that you require an extension, email Toby *well before the due date* to discuss this.

Please note that having assignments due around the same date for other subjects is not sufficient grounds to grant an extension. It is the responsibility of individual students to ensure that, if they have a cluster of assignments due at the same time, they start some of them early to avoid a bottleneck around the due date.

References

- [1] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. Bugs as deviant behavior: A general approach to inferring errors in systems code. *ACM SIGOPS Operating Systems Review*, 35(5):57–72, 2001.
- [2] Lanyue Lu, Andrea C Arpaci-Dusseau, Remzi H Arpaci-Dusseau, and Shan Lu. A study of linux file system evolution. In *11th USENIX Conference on file and storage technologies (FAST 13)*, pages 31–44, 2013.